MgDetailing

Documentación del proyecto web

Taller de detailing de autos, La Rioja, Argentina

19 de octubre de 2026

Versión final de esta etapa. Estado: panel con roles funcionando; todavía no publicada en internet.
Repositorio: GitHub MateoGimenez/MgDetail, rama main.
Escrito para el dueño del taller. Los términos técnicos se explican en el glosario del final.

Índice

1. Resumen ejecutivo 3

2. Cómo está organizado el proyecto 5

3. La web pública 8

4. El panel de administración 14

5. La base de datos 18

6. Seguridad 20

7. Avisos de turnos 25

8. Cómo usarlo 26

9. Cómo editar el contenido 28

10. Pendientes del dueño 30

11. Equipo de agentes 32

12. Glosario 33

1. Resumen ejecutivo

MgDetailing es la web del taller de detailing de autos de La Rioja, Argentina. Son dos cosas que se ven como una sola:

- **La web pública** (la "landing", la página de inicio que ve el cliente): explica los cinco servicios en palabras simples y deja pedir un turno con un formulario.
- **El panel de administración** (`/admin`): donde el equipo del taller ve las solicitudes de turno, las confirma, las marca como completadas, las cancela o las borra, y le escribe al cliente por WhatsApp. Lo que cada persona puede hacer depende de su rol: superadmin, admin o empleado. El superadmin además maneja la lista de usuarios.

Cómo está hecha. El frontend (lo que se ve) usa React 19 y Vite. El backend (el servicio que guarda los datos y controla los permisos) usa Node 24, Express 5 y TypeScript. Los datos y el login viven en **Supabase** (una base Postgres en la nube, con login propio de Supabase Auth). El diseño se llama "cartel fileteado": fondo grafito, rojo y fileteado, con el logo real y un monograma 3D que gira al pasar el mouse.

Estado final al 19 de octubre de 2026

Tema	Estado	Detalle
Web pública	Lista	Falta cargar datos del taller, fotos reales y precios
Panel de turnos	**Listo y funcionando**	El dueño confirmó que le funciona perfecto
Login con Supabase Auth y roles	**Listo**	Superadmin, admin y empleado. El registro público está cerrado y el superadmin ya está creado
Pantalla de Usuarios	**Lista**	El superadmin activa usuarios, les cambia el rol y los desactiva
Base de datos	**Lista en Supabase**	SQLite se eliminó. Las 6 migraciones están aplicadas
Auditorías de seguridad	**3 hechas**	Sin hallazgos críticos ni altos. Los 4 medios de la tercera ya están corregidos
Avisos de turnos	Desactivados	Telegram, WhatsApp Cloud API y CallMeBot están programados; falta que el dueño elija uno
Publicación	Sin publicar	Falta hosting y dominio

Números de calidad (verificados el 19/10/2026)

- Backend: **197 tests** automáticos, todos pasan. Frontend: **15 tests**, todos pasan.
- `npm audit` (revisión de librerías de terceros): **0 vulnerabilidades** (dato del orquestador).
- Advisors de Supabase (el chequeo automático de Supabase): sin avisos graves y **1 aviso** ("leaked password protection", una función de pago de Supabase; se cubre con contraseñas largas y únicas).
- Tres auditorías de seguridad hechas, sin hallazgos críticos ni altos.
- Código en GitHub (rama `main`): commits `3dc07ba`, `325237f` y `0507f37` (Supabase Auth, roles y usuarios).

> Qué le queda al dueño: todo es de puesta en marcha y de decisiones (limpiar el `.env`, aviso de privacidad, avisos de turnos, hosting y dominio, fotos y precios). Está en la sección 10.

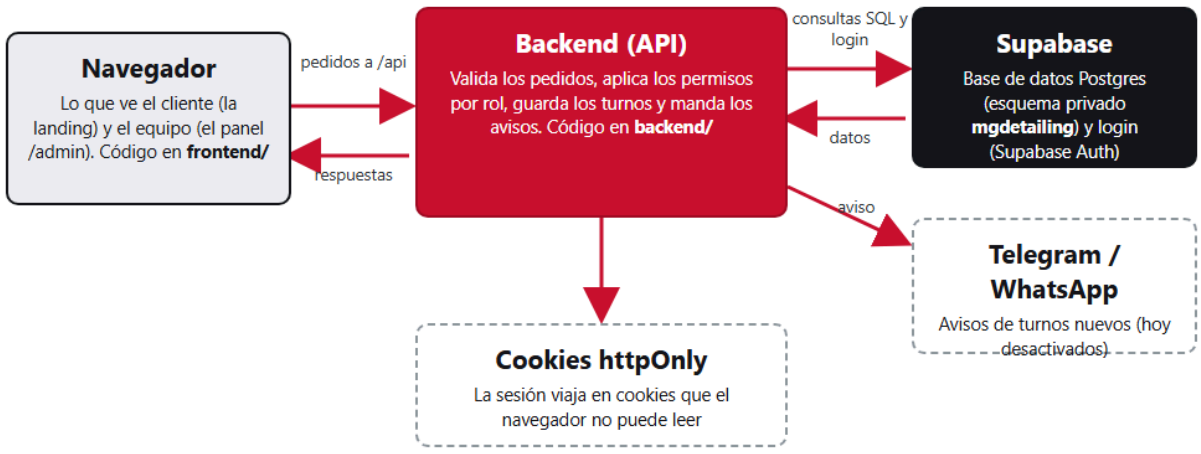
2. Cómo está organizado el proyecto

El proyecto vive en una carpeta con tres partes principales. Todo está en el repositorio de GitHub `MateoGimenez/MgDetail`, rama `main`.

Carpeta	Qué es
`frontend/`	Lo que ve el visitante y el panel. Hecho con React (librería para armar pantallas) y Vite (herramienta que prepara la web para publicarla).
`backend/`	La API: el servicio que guarda los turnos, controla el login y los permisos, y manda los avisos. Hecho con Node.js, Express y TypeScript.
`backend/supabase/migrations/`	Los archivos SQL que arman la base de datos en Supabase (el "plano" de las tablas).
`reportes/`	Informes del proyecto, capturas de pantalla y esta documentación.
`package.json` (raíz)	Los comandos para instalar, correr y verificar todo junto.

Diagrama de arquitectura

Quien visita la web o entra al panel habla **solo con el backend**, a través de la dirección `/api`. El navegador nunca habla con Supabase ni conoce ninguna clave: el backend hace de intermediario (esto se llama BFF, "backend for frontend").



Arquitectura: navegador, backend y Supabase.

Qué pasa cuando un cliente pide un turno

1. El cliente completa el formulario de la web (servicio, vehículo, fecha preferida y franja).
2. El navegador revisa los datos con las mismas reglas que el backend.
3. Llega al backend (`POST /api/bookings`). Un campo oculto (honeypot) y un límite de 5 pedidos cada 15 minutos por IP frenan a los bots.

4. El backend valida todo otra vez, lo guarda en Supabase con estado **pendiente** y responde al cliente.
5. Si hay un proveedor de avisos activo, manda un aviso al dueño (sin hacer esperar al cliente; si falla, el turno igual queda guardado).
6. El equipo ve la solicitud en el panel y la gestiona.

Qué hay dentro de cada carpeta

Frontend (`frontend/src/`)	Para qué sirve
`api/`	Único lugar que habla con la red (el contrato y las llamadas al backend).
`config/`	`sitio.ts` (datos del taller) y `enlaces.ts` (arma los links de WhatsApp, Instagram y Maps).
`data/`	Contenido editable: `servicios.ts`, `galeria.ts`, `preguntas.ts`.
`landing/`	Las secciones de la web pública y el formulario de turno.
`admin/`	El login y el panel de solicitudes.
`components/`	Logo, íconos y adornos de fileteado.
`lib/` y `styles/`	Fechas (hora de La Rioja), utilidades, fuentes y estilos base.

Backend (`backend/src/`)	Para qué sirve
`routes/`	Las direcciones de la API: salud, turnos, login, gestión de turnos y de usuarios.
`domain/`	Las reglas del negocio, incluido `permisos.ts` (qué puede hacer cada rol).
`services/`	Casos de uso: login con Supabase Auth, usuarios, alta de turno.
`repositories/postgres/`	El acceso a la base de datos (turnos y perfiles).
`middleware/`	Seguridad, anti-falsificación de pedidos, límites de uso, errores y archivos estáticos.
`notifications/`	Los avisos de turnos: Telegram, WhatsApp Cloud API y CallMeBot.
`schemas/` y `lib/`	Validación de datos de entrada, cookies, verificación de tokens.
`db/` y `scripts/`	Conexión a Postgres y el script que genera la contraseña del rol de la app.

Comandos principales (package.json)

Comando	Qué hace
`npm run instalar`	Instala todo lo necesario (raíz, backend y frontend).
`npm run dev`	Levanta backend (puerto 4000) y frontend (puerto 5173) para trabajar.
`npm run build`	Prepara la versión para publicar.
`npm start`	Arranca el backend ya compilado (después de `build`).
`npm test`	Corre los tests del backend y del frontend.

Comando	Qué hace
`npm run verificar`	Chequeo completo: tipos, lint, tests y build.

3. La web pública

La landing está pensada para quien nunca contrató detailing y llega desde Instagram con el celular: explica cada servicio sin jerga (y con honestidad sobre lo que un tratamiento logra y lo que no) y lleva siempre a pedir turno.

Secciones, en orden

#	Sección	Qué hace
1	Cabecera	Logo, menú y botón "Reservá tu turno". En celular, menú desplegable.
2	Portada (hero)	Título "Hasta el último detalle.", frase y un formulario corto: tipo de vehículo y servicio.
3	Servicios	Los cinco servicios con qué incluye, duración y precio ("Consultá" si todavía no hay precio).
4	Cómo trabajamos	Los pasos del proceso.
5	Trabajos	Galería de fotos y comparador antes/después. Hoy dice "Fotos reales muy pronto".
6	Beneficios	Puntos a favor del taller.
7	Frase y monograma	"Lo bueno se hace con tiempo." con el monograma 3D.
8	Preguntas frecuentes	Diez preguntas y respuestas.
9	Reserva	El formulario completo de turno.
10	Contacto y pie	WhatsApp, Instagram, dirección, horarios y mapa. Lo que no esté cargado se oculta solo.

Los servicios

Las descripciones, lo que incluye cada servicio y las duraciones son una **propuesta sin confirmar**: están marcadas `COMPLETAR` en el código hasta que el taller las apruebe. Los precios están todos en "Consultá".

Servicio	Duración propuesta
Lavado Básico	1 a 1,5 h
Lavado Premium (destacado)	3 a 4 h
Pulido	Alrededor de 1 día
Encerado	2 a 3 h
Restauración de ópticas	1,5 a 2 h

El formulario de turno

Pide nombre, teléfono, email (opcional), tipo de vehículo (auto, camioneta o moto), modelo (opcional), servicio, fecha preferida (de hoy a 60 días, con hora de La Rioja) y franja (mañana o tarde). No es una agenda en tiempo real: el taller confirma después.

Diseño: "el cartel fileteado"

El concepto imita los carteles y carrocerías pintados a mano en Argentina: filete rojo de un solo trazo con volutas en espiral, sobre un panel de color grafito. La web se ve completa sin fotos y deja lugar para las fotos reales cuando lleguen. El rojo es el único color de marca; el ornamento enmarca pero nunca va detrás de un texto.

Color	Código	Para qué se usa
Rojo filete	#e42c42	Líneas de adorno, foco de campos, lo elegido
Rojo chapa	#c8102e	Botón principal
Grafito	#14171a	Fondo de la landing
Blanco	#f7f8f9	Texto principal
Gris imprimación	#eceef1	Fondo del área de trabajo del panel (se usa de día en el taller)

Tipografía: "Señalética de ruta"

Se eligió entre tres propuestas (más la anterior, que se descartó). Está terminada.

Uso	Fuente
Títulos	Barlow Condensed 800: letra condensada pesada, como la de los carteles de ruta; mismo ángulo y peso que el monograma del logo.
Acento (frase y "Detailing")	Kaushan Script: letra de pincel, igual al "Detailing" del logo.
Texto corrido	Archivo: sobria y fácil de leer de cerca.

Las fuentes se sirven desde la propia web (no desde Google Fonts) y pesan 92,1 KB en total. **Regla del cromo escaso:** el efecto metálico se usa solo en el logo, el título principal y la frase; los títulos de sección van en blanco liso.

Marca: logo, monograma y archivos

El **monograma "Mg"** es un video 3D de 1080 x 1080 píxeles. En la banda "Lo bueno se hace con tiempo." **gira 180°** al pasar el mouse y muestra del otro lado el logo 3D completo. En celular, un toque lo hace girar y vuelve solo a los 3 segundos. Se maneja con teclado, y si la persona tiene activada la opción de "menos movimiento" del sistema, no gira y se muestra un cuadro fijo.

Archivo	Dónde está	Uso
Logo 3D, avatar, planos para fondo claro y oscuro, video del monograma	`frontend/public/fotos/MgDetailing-*`	Originales. No se tocan.
`logo-fondo-oscuro.png`	`frontend/public/marca/`	Logo en cabecera, pie y panel
`monograma-poster.webp`, `monograma-dorso.webp`	`frontend/public/marca/`	Cuadro fijo y dorso del monograma
`og.jpg`	`frontend/public/marca/`	Imagen al compartir el link en redes
`favicon-48.png`, `apple-touch-icon.png`	`frontend/public/marca/`	Ícono de la pestaña y del celular

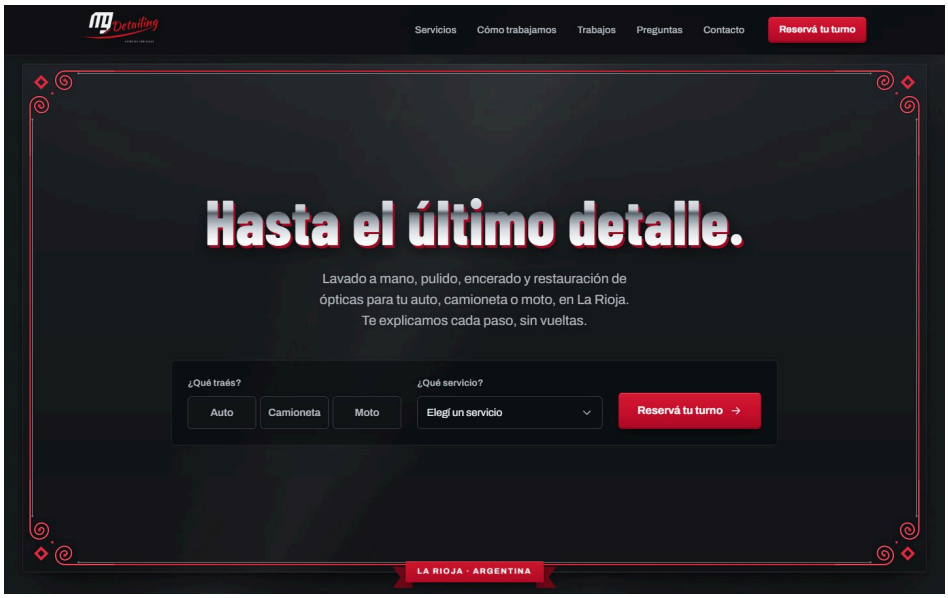
Las versiones livianas se regeneran con `npm --prefix frontend run marca`.

Rendimiento y accesibilidad

Archivo	Comprimido	Cuándo se carga
JS de la landing	106,6 KB	Siempre
CSS	9,3 KB	Siempre
JS del panel	5,2 KB	Solo al entrar a `/admin`

- El video del monograma se carga solo cuando se ve en pantalla, pero se sirve el original de **1,95 MB**; si pesa mucho en celular conviene comprimirlo.
- Accesibilidad (objetivo WCAG 2.1 AA): campos con etiqueta, errores que el lector de pantalla anuncia, foco visible, comparador antes/después manejable con teclado y respeto de "menos movimiento". Hay reglas automáticas de accesibilidad en el lint.
- **No se hizo una auditoría de accesibilidad completa** con una herramienta de contraste: lo anterior es lo que está en el código.
- Para buscadores: datos estructurados del negocio, etiquetas para compartir y `canonical`. Estas dos últimas solo se emiten cuando `url` en `sitio.ts` tenga la dirección https real.

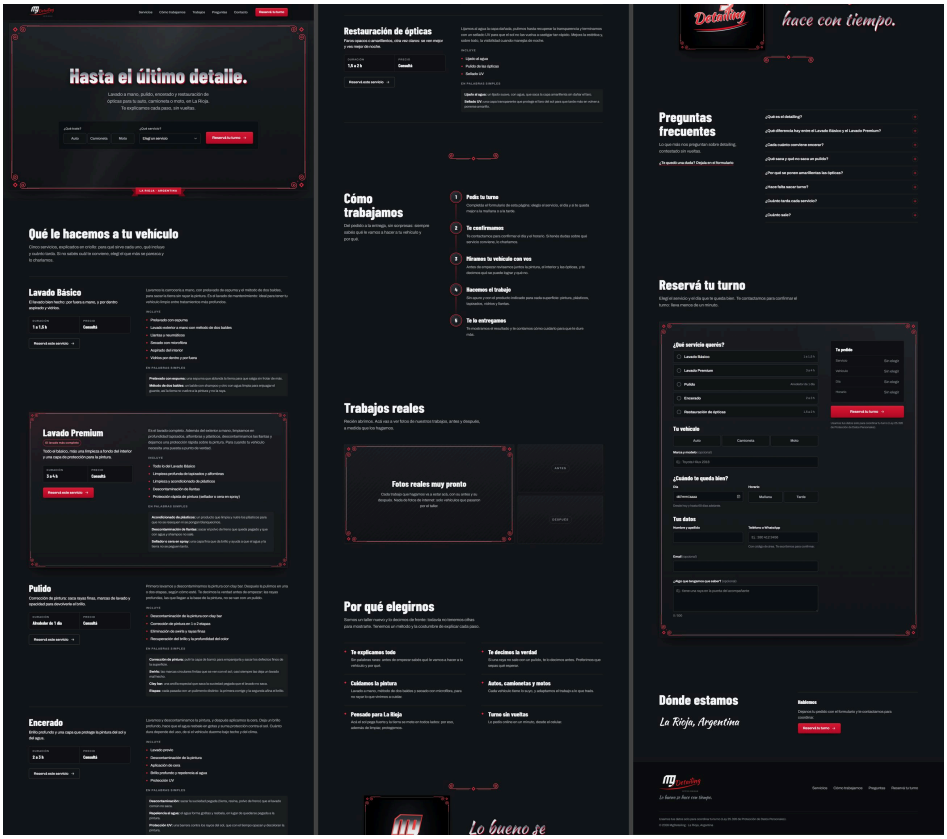
Capturas



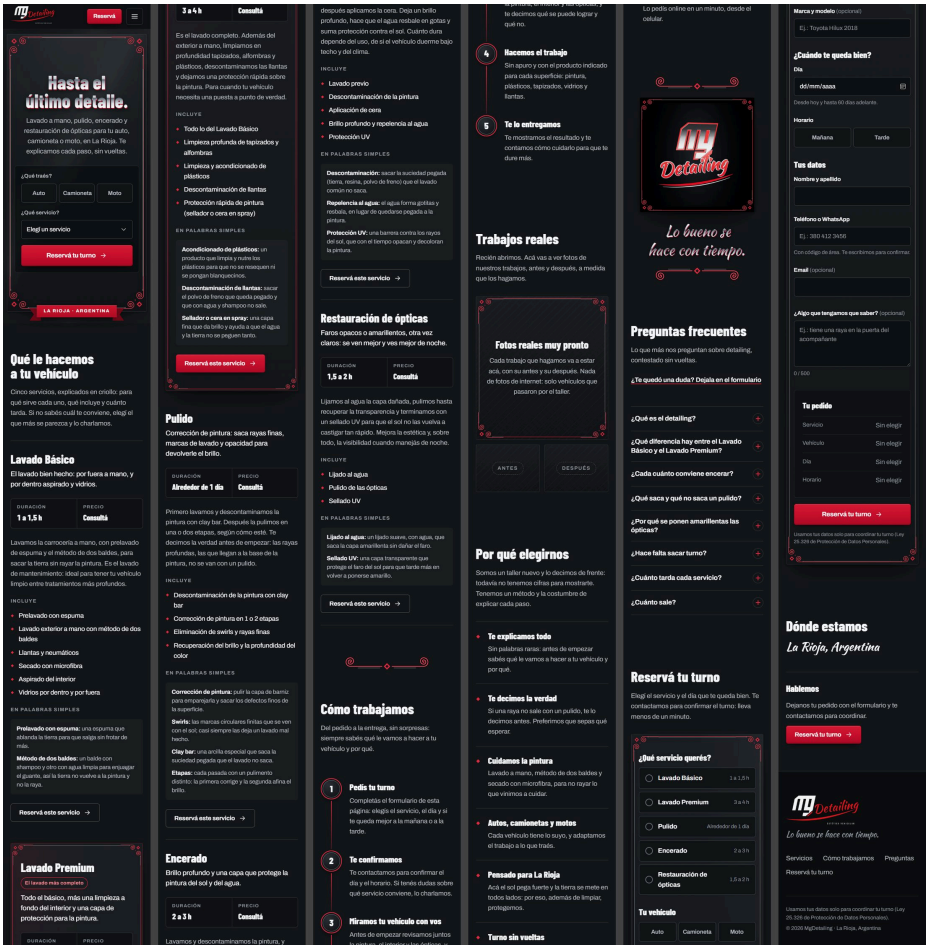
Inicio en escritorio.



Inicio en celular.



Página completa en escritorio (cortada en tres tiras, de izquierda a derecha).



Página completa en celular (seis tiras, de izquierda a derecha).



Comparación de tipografías: la anterior, las tres propuestas y el logo real. Se eligió la 1, "Señalética de ruta".



El monograma girado, mostrando el logo 3D, junto a la frase "Lo bueno se hace con tiempo."

4. El panel de administración

Se entra en `/admin` con email y contraseña. No hay registro público: cada usuario lo crea el dueño en Supabase. El panel usa el mismo estilo de marca pero con el área de trabajo clara, para leerlo bien a la luz del taller.

Turnos y estados

Las solicitudes se ven en pestañas: **Pendientes, Confirmadas, Completadas, Canceladas y Todas**. Cada tarjeta muestra servicio, vehículo, fecha y franja, teléfono, email, comentario y cuándo se pidió.

Estado	Botones principales	Qué significa
Pendiente	Confirmar, Cancelar	Entró recién; el taller todavía no respondió.
Confirmado	Marcar completado, Cancelar	El taller aceptó el turno.
Completado	(selector de estado)	El trabajo se hizo.
Cancelado	Volver a pendiente	El turno no se hará.

En todas las tarjetas hay además un selector de estado, el botón **Eliminar** (pide confirmación) y **Escribir por WhatsApp**.

WhatsApp al cliente

El botón **Escribir por WhatsApp** abre un chat con el cliente con el mensaje ya armado. El sistema normaliza el teléfono que escribió el cliente (por ejemplo "0380 15-4123456" pasa a 5493804123456). Si el teléfono no se puede interpretar, el botón no aparece, pero el turno nunca se rechaza por eso.

Roles y permisos

La regla vive en el backend (`backend/src/domain/permisos.ts`): la pantalla solo esconde botones, pero quien decide es el servidor. Lo que no está permitido responde "403 prohibido".

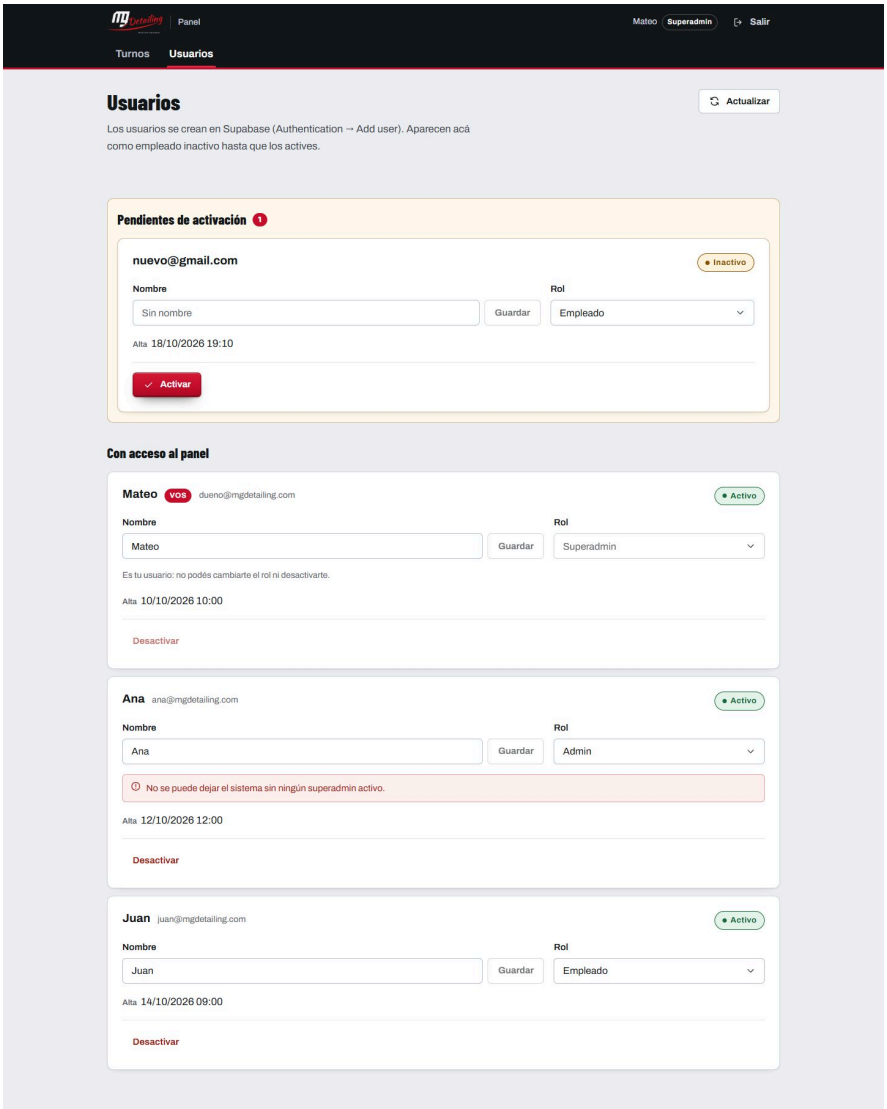
Rol	Ver turnos	Cambiar estado	Borrar turnos	Gestionar usuarios
superadmin	Sí	A cualquier estado	Sí	Sí
admin	Sí	A cualquier estado	Sí	No
empleado	Sí	Solo a "completado"	No	No

La pantalla de Usuarios (solo superadmin)

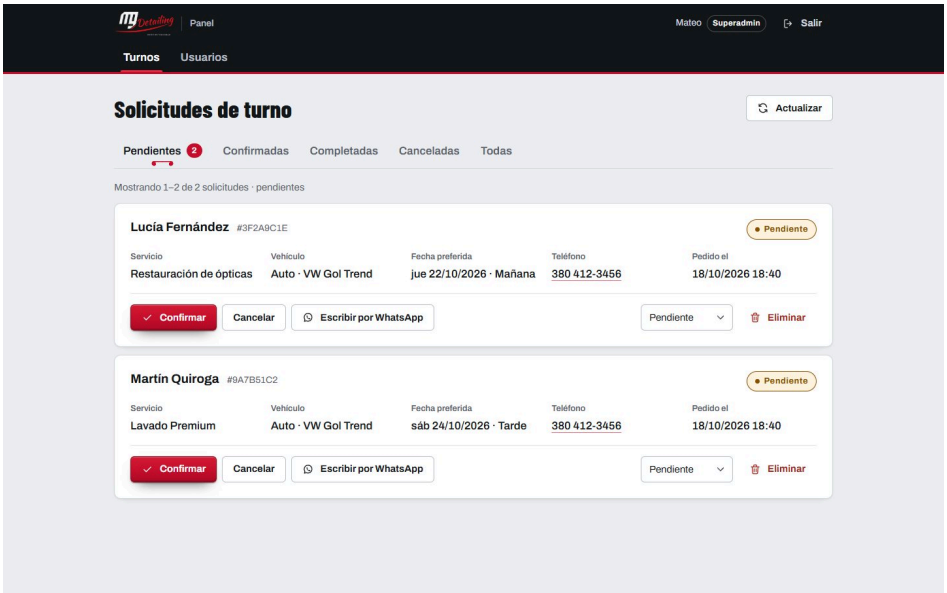
El superadmin ve una pestaña **Usuarios** junto a **Turnos**. Los usuarios se crean en Supabase (*Authentication > Add user*) y aparecen ahí en un recuadro **"Pendientes de activación"**. Desde la pantalla puede **activarlos**, **cambiarles el rol**, **editarles el nombre** y **desactivarlos**. En el backend, la lista es `GET /api/admin/usuarios` y los cambios `PATCH /api/admin/usuarios/:id`. Reglas de seguridad:

- Los usuarios nuevos nacen como **empleado** e **inactivos**: no entran hasta que un superadmin los active.
- Nadie puede cambiarse su propio rol ni desactivarse a sí mismo (la pantalla muestra "Es tu usuario: no podés cambiarte el rol ni desactivarte").
- Nunca puede quedar el sistema sin un superadmin activo. La base de datos lo garantiza al cambiar el rol, al desactivar y también al borrar, incluso con cambios simultáneos.
- Desactivar a alguien, cambiarle el rol o cerrarle la sesión tiene efecto en el pedido siguiente, aunque su sesión no haya vencido.
- El empleado ve solo los turnos y el botón **Marcar completado** (más **Escribir por WhatsApp**); no ve **Confirmar**, **Cancelar**, **Eliminar**, ni el selector de estado, ni la pestaña Usuarios.

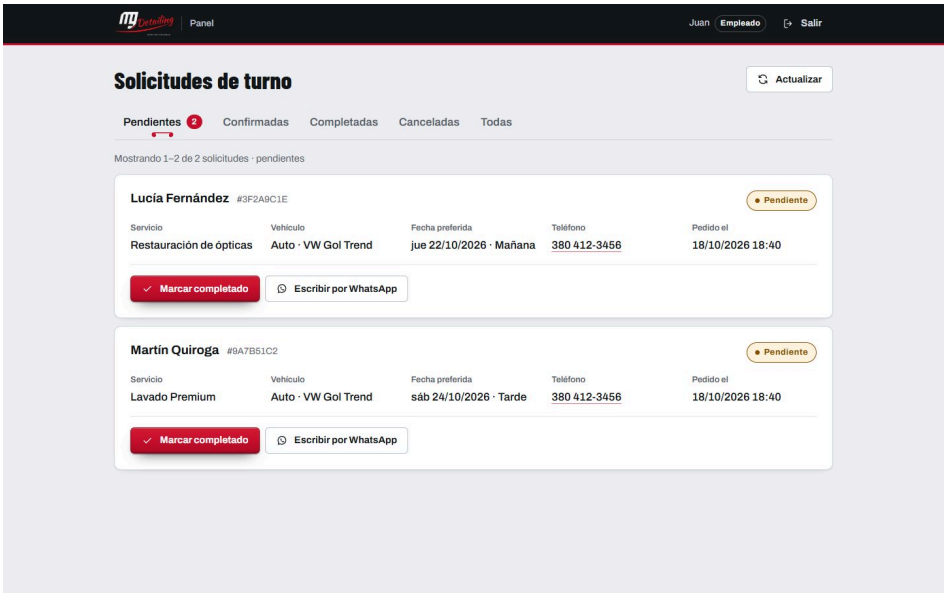
> **LISTO**: la pantalla de Usuarios y la adaptación del panel a los roles están terminadas y el dueño confirmó que el panel funciona perfecto. Las capturas de abajo se sacaron con **datos simulados** (nombres, emails y turnos inventados), no con clientes ni usuarios reales.



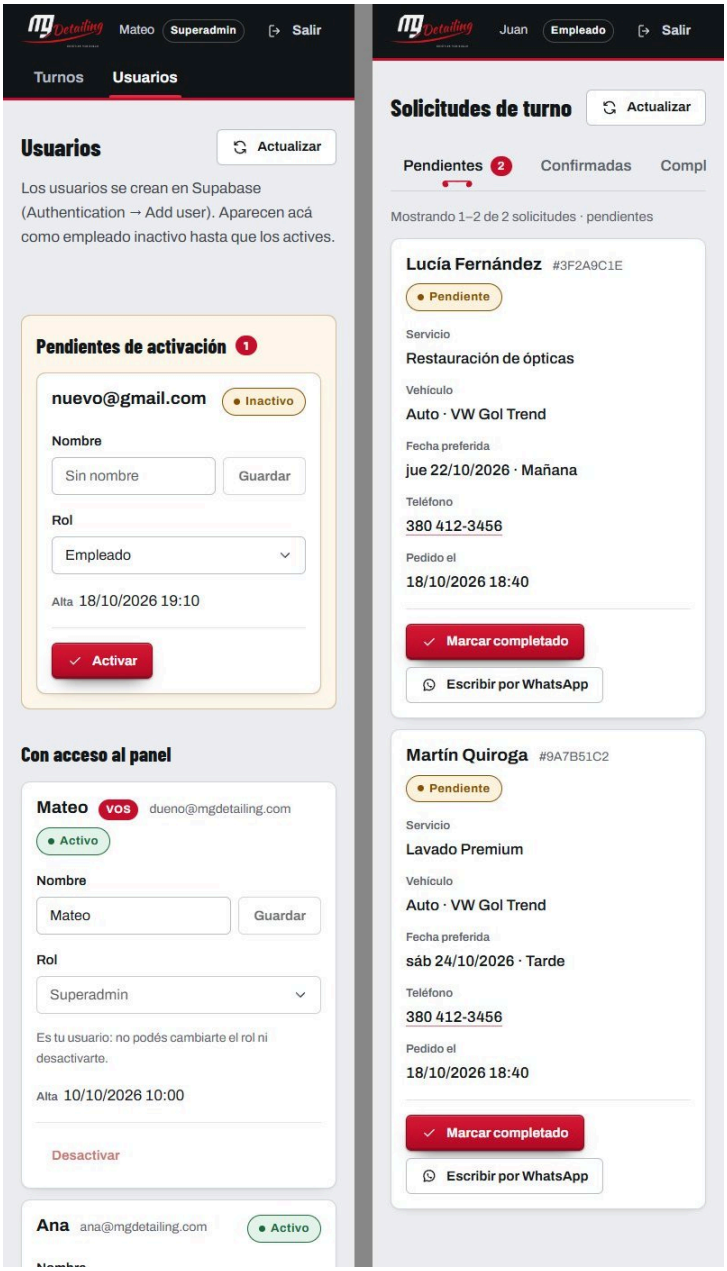
Pantalla de Usuarios del superadmin (datos simulados): usuario pendiente de activación, cambio de rol y desactivar. Se ve el aviso "No se puede dejar el sistema sin ningún superadmin activo".



Turnos vistos por el superadmin (datos simulados): Confirmar, Cancelar, Escribir por WhatsApp, selector de estado y Eliminar.



Turnos vistos por un empleado (datos simulados): solo Marcar completado y Escribir por WhatsApp.



El panel en celular (datos simulados): a la izquierda Usuarios (recortada) y a la derecha los turnos de un empleado.

5. La base de datos

Todo se guarda en **Supabase (Postgres)**, dentro de un esquema propio llamado `mgdetailing` (un "esquema" es una carpeta de tablas dentro de la base). SQLite, la base de archivo que se usaba al principio, **fue eliminada**: ahora hay un solo motor.

Las tablas

Tabla	Para qué sirve	Datos principales
`servicios`	Catálogo de los 5 servicios. Un servicio que se deja de ofrecer se desactiva, no se borra.	id (por ejemplo `lavado-basico`), nombre, duración estimada, precio desde (vacío = "Consultá"), activo, orden
`bookings`	Las solicitudes de turno.	id público (UUID), nombre, teléfono, email, tipo de vehículo, modelo, servicio, fecha preferida, franja, comentario, estado, fechas
`roles`	Los 3 roles del panel.	superadmin, admin, empleado, con su descripción
`perfiles`	Un perfil por cada usuario de Supabase Auth: qué rol tiene y si puede entrar.	id (el del usuario), email, nombre, rol, activo

- `bookings` usa un id público tipo UUID (no un número correlativo) para que nadie pueda adivinar cuántos turnos hay.
- Los valores permitidos (tipo de vehículo, franja, estado) los controla la propia base con restricciones `CHECK`, además de la API.
- Cada servicio del código tiene que existir también en la base: si se agrega uno, va con una migración nueva.
- Los usuarios y sus contraseñas están en Supabase Auth (esquema `auth`), no en las tablas de arriba. El backend **no guarda contraseñas**.

Seguridad en la base

- **Esquema privado:** `mgdetailing` y `mgdetailing_private` no están expuestos por la Data API de Supabase (la puerta que publica tablas por internet). La web nunca habla con Supabase directo.
- **Rol de mínimo privilegio:** el backend se conecta como `mgdetailing_app`, no como `postgres`. Solo puede leer servicios y roles; leer, crear y borrar turnos (cambiar solo su estado); y leer perfiles (cambiar solo rol, activo y nombre). No puede crear ni borrar tablas, ni tocar `auth` ni `public`.
- **RLS (Row Level Security, seguridad por fila)** activado en las 4 tablas, con reglas solo para `mgdetailing_app`. Los roles públicos de Supabase (`anon`, `authenticated`, `service_role`) no tienen permisos ni reglas: aunque alguien expusiera el esquema por error, no vería ninguna fila.
- **TLS verificado:** la conexión va cifrada y el backend comprueba el certificado de Supabase (necesita el archivo `DATABASE_SSL_CA_FILE`).
- **Topes:** cada consulta tiene un máximo de 10 segundos, y el pool de conexiones es chico (5).

- **Región `us-east-1` (EE.UU.):** los datos de los clientes se guardan fuera de Argentina. Ver "Datos personales" en la sección 6.

Migraciones

Una migración es un archivo SQL que cambia la base. Viven en `backend/supabase/migrations/`, **no las aplica el backend** (las aplica el dueño o la CLI de Supabase) y nunca se edita una ya aplicada: los cambios van en un archivo nuevo. Al arrancar, el backend verifica que estén aplicadas y, si falta algo, no arranca y dice qué.

Archivo	Qué hace	Estado
`20261018211151_esquema_inicial`	Crea el esquema, las tablas de turnos y servicios y los permisos base.	Aplicada
`20261018211156_catalogo_servicios`	Carga los 5 servicios.	Aplicada
`20261018211232_endurecer_rls_auto_enable`	Quita un permiso de una función de Supabase que el chequeo marcaba.	Aplicada
`20261018213342_rol_app_minimo_privilegio`	Crea el rol `mgdetailing_app` con permisos mínimos.	Aplicada
`20261018234300_perfiles_roles_supabase_auth`	Crea `roles` y `perfiles`, el disparador (trigger) de `auth.users`, la protección "nunca sin superadmin" (al cambiar y al borrar), la función `sesion_vigente` con el tope de 12 horas y el esquema privado.	Aplicada
`20261018234301_eliminar_admins_sessions`	Borra las tablas viejas `admins` y `sessions` (estaban vacías).	Aplicada

> **LISTO**: el dueño aplicó las dos migraciones nuevas desde el SQL Editor de Supabase, y el backend nuevo arrancó y funciona. Una migración nueva se aplica igual: pegar el archivo en el SQL Editor (o usar la CLI de Supabase) y después revisar los advisors. El README del backend todavía dice que esas dos están pendientes: quedó desactualizado.

6. Seguridad

Resumen para el dueño. La seguridad está bien para un sitio de poco tráfico. Las tres auditorías no encontraron hallazgos críticos ni altos, y lo que se probó con ataques reales resistió. Los 4 hallazgos medios de la última auditoría ya están corregidos. El riesgo que queda es bajo y depende sobre todo de **cómo se opere** la web: la configuración del proxy, la fortaleza de las contraseñas, cuánto tiempo se guardan los datos y qué terceros reciben los avisos.

Qué auditorías hubo

Auditoría	Quién	Resultado
1. Seguridad de toda la web (informe 05 del 17/10/2026), con el sistema anterior: ataques reales contra un servidor de prueba	Agente de ciberseguridad (Opus)	13 hallazgos (S1 a S13), ninguno crítico ni alto.
2. Modelado de la base de datos en Supabase y advisors	Agente de ciberseguridad (Opus)	Sin avisos de seguridad en ese momento.
3. Login nuevo con Supabase Auth, roles y usuarios	Agente de ciberseguridad (Opus)	13 hallazgos (A1 a A13): 4 medios y 3 bajos corregidos; el resto, aceptados. Ver la tabla de abajo.

> El informe 05 se hizo con el sistema anterior (SQLite y login propio). Los hallazgos S1 a S13 se actualizaron contra el código y los README actuales. El detalle de la auditoría 3 sale del resumen del orquestador y del código; no hay un informe suyo en `reportes/`.

Tercera auditoría (login nuevo): hallazgos A1 a A13

ID	Riesgo	Qué pasaba	Estado
A1	Medio	Se podía hacer perder tiempo y recursos al servidor mandando cookies de refresco falsas una y otra vez (DoS del refresh).	**Corregido.**
A2	Medio	Una sesión abierta se renovaba para siempre: en el plan Free el refresh token de Supabase no vence.	**Corregido:** tope absoluto de **12 horas** desde el login, aunque se siga refrescando.
A3	Medio	Se podía saber qué usuarios existen pero están inactivos, por la respuesta del login.	**Corregido:** el login devuelve el mismo 401 genérico también para usuarios inactivos.
A4	Medio	El registro público de Supabase estaba abierto.	**Corregido:** el dueño lo cerró.
A5	Bajo	No se distinguía "sin acceso al panel" de "rol sin permiso".	**Corregido:** `NO_ACCESS` (usuario sin acceso al panel) es distinto de `FORBIDDEN` (el rol no puede hacer eso).

ID	Riesgo	Qué pasaba	Estado
A6	Bajo	Se podía borrar al último superadmin.	**Corregido:** la base lo impide también al borrar, no solo al editar.
A7	Bajo	Una clave secreta puesta por error en el `.env` pasaba sin aviso.	**Corregido:** el servidor avisa si hay claves secretas en el `.env` (ver la sección 10).
A8 a A13	Info	Observaciones menores.	Aceptados. Su detalle no está en el repositorio.

Advisors de Supabase: quedó **1 aviso** ("leaked password protection", que revisa si una contraseña apareció en filtraciones conocidas). Es una función de pago de Supabase; la recomendación es usar contraseñas largas y únicas.

Configuración recomendada de Supabase Auth

Resumen del orquestador (el README del backend no la detalla toda). Se configura en el panel de Supabase:

- Registro de usuarios nuevos **cerrado** (ya hecho).
- Contraseñas de **12 caracteres o más**, con requisitos de complejidad.
- **Detección de reuso del refresh token** activada.
- Duración del JWT de **3600 segundos o menos**.
- Rotar las claves de firma dejando la anterior en espera (standby).
- En el backend, **solo la clave publishable**.
- MFA (segundo factor) opcional, para más adelante.

Hallazgos de la primera auditoría (S1 a S13) y su estado

ID	Riesgo	Qué pasaba	Estado hoy
S1	Medio	Con `TRUST_PROXY=1` y el puerto de Node accesible desde afuera, se podía falsear la IP y saltear el límite de intentos de login.	Corregido en la documentación. VPS con Caddy: `HOST=127.0.0.1`, `TRUST_PROXY=loopback` y puerto 4000 cerrado. `TRUST_PROXY=1` solo en un PaaS. Vigente al elegir hosting.
S2	Bajo	Un archivo estático inexistente devolvía 200 con HTML.	Corregido: ahora 404. Hay test.
S3	Bajo	El panel podía aparecer en buscadores.	Corregido: `X-Robots-Tag: noindex, nofollow` en `/admin`.
S4	Bajo	Archivos auxiliares de la base SQLite legibles por otros usuarios del sistema.	**Ya no aplica**: SQLite fue eliminado.

ID	Riesgo	Qué pasaba	Estado hoy
S5	Bajo	Límite de intentos de login por IP y un solo factor.	Mitigado: el login ahora es de Supabase Auth, que suma sus propios límites; el backend mantiene 5 fallos cada 15 minutos; hay tope de sesión de 12 h. Seguir con contraseñas largas y únicas; MFA queda para más adelante.
S6	Bajo	La cookie de sesión no tiene el prefijo `__Host-`.	Pendiente, opcional. Las cookies nuevas son `mg_at` (`Path=/api`) y `mg_rt` (`Path=/api/auth`), `HttpOnly` y `SameSite=Strict`.
S7	Bajo	Los originales de marca (unos 2,4 MB) se publican en `frontend/public/fotos/`.	Pendiente: moverlos fuera de `public/`.
S8	Info	No se cancela el cuerpo de la respuesta del proveedor de avisos.	Aceptado.
S9	Info	El nombre y el comentario del cliente llegan en texto plano al aviso.	Aceptado. Confiar solo en el link `wa.me` que arma el sistema.
S10	Info	La consulta previa (preflight) de CORS responde 200 sin cabeceras.	Aceptado.
S11	Info	HSTS dura 180 días.	Aceptado. Subir a un año con el dominio estable.
S12	Info	Una ruta inexistente de la web responde 200 con "No encontrada".	Aceptado. La API sí da 404.
S13	Info	Un número de teléfono de ejemplo en los archivos de ejemplo y el README.	Aceptado. Conviene reemplazarlo por uno ficticio.

Qué se probó y no tuvo hallazgos

- **Dependencias** (`npm audit`): 0 vulnerabilidades.
- **Tests:** 197 del backend y 15 del frontend, todos verdes.
- Pedido desde otro sitio: rechazado (403). Tipo de contenido incorrecto: 415. Campos extra: 400. Cuerpo demasiado grande: 413.
- Intentos de pedir archivos fuera de la carpeta pública (`../`, `/.env`, `/.git`): 404.
- `/api/admin` sin sesión: 401. Conexiones lentas (Slowloris): se cortan a los 17 segundos aproximadamente.
- Avisos: direcciones fijas (sin SSRF), tiempo máximo de 5 s y sin secretos en los logs.
- Frontend: sin `dangerouslySetInnerHTML`, enlaces externos con `noopener` y nada sensible en `localStorage`.
- Secretos en el repositorio: ninguno versionado.
- Logs sin datos personales: solo ids, nunca emails, teléfonos ni tokens.

Cómo funciona el login nuevo (Supabase Auth, modo BFF)

- El navegador manda email y contraseña al backend; el backend le pregunta a Supabase y **verifica la firma del token** (algoritmo ES256, emisor, audiencia y vencimiento fijados).
- Los tokens viajan en **cookies** `HttpOnly` **y** `SameSite=Strict` (`Secure` en producción): JavaScript no puede leerlas y no se mandan desde otros sitios. El navegador nunca ve una clave.
- En **cada pedido** al panel se comprueba que la sesión siga viva en Supabase y que el perfil esté activo, así que cerrar sesión, desactivar o cambiar de rol es inmediato.
- Cookies: `mg_at` (el token de acceso, solo para `/api`) y `mg_rt` (el de refresco, solo para `/api/auth`). Una sesión dura **como máximo 12 horas desde el login**, aunque se siga usando.
- Errores distintos: `NO_ACCESS` (tu usuario no tiene acceso al panel) y `FORBIDDEN` (tu rol no puede hacer eso). El login devuelve el mismo error genérico si el usuario no existe, si la contraseña está mal o si está inactivo.
- El backend usa solo la clave **publishable** de Supabase. La clave secreta no se usa y el servidor se niega a arrancar si se la pone en su lugar.
- Además: protección contra pedidos falsificados (chequeo de `Origin` y `Content-Type`), límites de pedidos, cabeceras de seguridad y errores sin detalles internos.

Checklist de producción

Versión actualizada al sistema con Supabase (ya no hay `JWT_SECRET` ni `DB_PATH`). Algunos puntos ya están hechos; se dejan para tener la lista completa.

- ☐ `NODE_ENV=production`.
- ☐ `FRONTEND_ORIGIN` exacto, con https y sin barra final (con y sin `www` si corresponde, separados por coma).
- ☐ `SERVE_FRONTEND=true`.
- ☐ `SUPABASE_URL` y `SUPABASE_PUBLISHABLE_KEY` cargadas (la publishable, nunca la secreta).
- ☐ `DATABASE_URL` con el rol `mgdetailing_app` y `DATABASE_SSL_CA_FILE` con el certificado de Supabase.
- ☐ VPS con Caddy: `HOST=127.0.0.1`, `TRUST_PROXY=loopback` y firewall con el puerto 4000 cerrado. PaaS: `HOST=0.0.0.0` y `TRUST_PROXY=1`.
- ☐ Registro público de Supabase **desactivado** (ya hecho).
- ☐ Un proveedor de avisos elegido, o `WHATSAPP_PROVIDER=none`.
- ☐ Archivo `.env` con permisos restringidos y fuera del repositorio.
- ☐ Monitor externo sobre `/api/health`.
- ☐ Vigilar ráfagas de errores 401 y 429 en el login, spam de turnos, errores 500 y el vencimiento del certificado HTTPS.
- ☐ `npm audit` en cada publicación.
- ☐ Backups: el plan **Free** de Supabase no hace copias que se puedan descargar o restaurar. Hacer un `pg_dump` propio (por ejemplo una vez por semana), guardarlo cifrado y fuera de la PC, y probar una restauración al menos una vez.

- ☐ `.env` limpio: sin `SUPABASE_SECRET_KEY`, `JWT_SECRET` ni `DB_DRIVER` (el servidor avisa si quedan).
- ☐ Un proyecto Free de Supabase **se pausa tras una semana sin actividad**: si el panel lo muestra pausado, se reactiva desde ahí.
- ☐ Si hay sospecha de filtración: rotar la contraseña de `mgdetailing_app`, las claves de Supabase y el token de Telegram.

Datos personales (Ley 25.326)

> Esto no es asesoramiento legal: son propuestas para revisar con un abogado.

Qué datos se guardan: nombre, teléfono, email, tipo de vehículo, modelo, comentario y el turno. Son datos personales.

- **Aviso de privacidad:** el formulario todavía no tiene uno completo. El informe 05 propone un texto: "Usamos tus datos (nombre, teléfono, email, vehículo y comentario) solo para coordinar tu turno. Los guarda MgDetailing, [responsable], La Rioja. Para avisarnos de tu pedido pasan por [Telegram/WhatsApp]. Los borramos [N] meses después del turno. Podés pedir verlos, corregirlos o borrarlos escribiéndonos. La Agencia de Acceso a la Información Pública es el órgano de control de la Ley 25.326." Falta completar responsable, plazo y canal.
- **Transferencia internacional:** Supabase guarda los datos en EE.UU. (`us-east-1`). Ese país no figura entre los de protección adecuada para la AAIP (art. 12 de la Ley 25.326), así que hay que apoyarse en el **consentimiento del cliente** (mencionarlo en el aviso de privacidad) y en el **DPA** (acuerdo de tratamiento de datos) de Supabase.
- **Plazo de guardado:** hoy no hay política. Propuesta: borrar turnos completados o cancelados a los 6 meses y los pendientes viejos a los 3. Está propuesto, **no implementado**.
- **Terceros:** si hay avisos activos, los datos del cliente pasan por el proveedor (Telegram, Meta o CallMeBot).
- **Backups:** borrar un turno no lo saca de las copias hasta que estas roten; conviene decirlo en el aviso.

7. Avisos de turnos

Cuando entra un turno, el dueño puede recibir un aviso con el servicio, la fecha, el cliente, su teléfono, un link `wa.me` para escribirle y el link al panel (si está cargada `PUBLIC_URL`). Se manda **después** de guardar el turno y **sin esperar**: si el proveedor falla o tarda más de 5 segundos, el cliente igual recibe su confirmación.

> Estado: los avisos están **desactivados** (`WHATSAPP_PROVIDER=none` por defecto) hasta que el dueño configure uno. Los tres proveedores están programados y con tests con respuestas simuladas, pero **ninguno se probó contra el servicio real**.

Opciones

Opción	Costo	Qué hace falta	Riesgo / observaciones
Telegram (recomendada para arrancar)	Gratis	Un bot creado con @BotFather y el chat id	Oficial. El aviso llega a Telegram, no a WhatsApp.
WhatsApp Cloud API (Meta)	Unos US$1 por mes (cifra a verificar en la tarifa de Meta)	Cuenta de Meta Business, número propio y plantilla aprobada	Oficial, pero con más pasos. No probado contra Meta.
CallMeBot	Gratis	Número del dueño y una apikey	No oficial, de terceros, sin garantías. Hoy no tiene plazas. Evitar para datos personales.

Pasos para Telegram

1. En Telegram, buscá a `@BotFather` (el oficial, con tilde azul), mandale `/newbot` y elegí un nombre y un usuario que termine en `bot`. Te devuelve un **token**: es una clave, no la compartas.
2. Abrí tu bot nuevo y mandale `/start` y cualquier mensaje. Sin esto el bot no puede escribirte.
3. En el navegador abrí `https://api.telegram.org/bot<TOKEN>/getUpdates` (reemplazá `<TOKEN>` por el tuyo) y buscá `"chat":{"id":...`: ese número es tu **chat id**.
4. En `backend/.env` poné las tres líneas del recuadro de abajo (valores de ejemplo).
5. Reiniciá el servidor: en el log tiene que aparecer "Avisos de turnos activados (telegram)". Hacé un turno de prueba y comprobá que llega.

```
WHATSAPP_PROVIDER=telegram
TELEGRAM_BOT_TOKEN=123456789:AAH-ejemplo
TELEGRAM_CHAT_ID=123456789
```

8. Cómo usarlo

Instalar y correr

- Hace falta **Node.js 24 o superior** (se instala desde nodejs.org). npm viene con Node.

```
npm run instalar   # instala todo
npm run dev        # backend en :4000 y frontend en :5173
```

Abrí la web en `http://localhost:5173` y el panel en `http://localhost:5173/admin`. Usá siempre `localhost`, no `127.0.0.1`: el backend rechaza los pedidos que vienen de otro origen (error 403).

El archivo backend/.env

Sin él el panel no funciona (necesita Supabase). Se arma copiando `backend/.env.example` a `backend/.env`. **Es secreto: nunca va al repositorio.** Ejemplo con valores ficticios:

```
SUPABASE_URL=https://tu-proyecto.supabase.co
SUPABASE_PUBLISHABLE_KEY=sb_publishable_xxxxxxxx
DATABASE_URL=postgresql://mgdetailing_app:TU-CONTRASEÑA@host:5432/postgres
DATABASE_SSL_CA_FILE=./certs/supabase-ca.crt
FRONTEND_ORIGIN=http://localhost:5173
WHATSAPP_PROVIDER=none
```

La contraseña del rol `mgdetailing_app` se genera con `npm --prefix backend run clave-db:dev`, que imprime la sentencia que hay que pegar en el SQL Editor de Supabase. Las variables viejas (`DB_PATH`, `JWT_SECRET`, `DB_DRIVER`) ya no se usan: borralas (el servidor avisa si quedan). **No pongas una clave secreta (`sb_secret_...`, `SUPABASE_SECRET_KEY`) en el `.env`:** el backend no la necesita y también avisa si la encuentra.

Crear el primer usuario y promoverlo a superadmin

Esto se hace una sola vez. **Ya está hecho** para el dueño (migraciones aplicadas, registro público cerrado y superadmin creado); se deja por si hay que repetirlo en otro proyecto de Supabase.

1. **Cerrá el registro público.** En el panel de Supabase: *Authentication > Sign In / Providers > Email* y desactivá "Allow new users to sign up". Guardá.
2. **Creá el usuario.** *Authentication > Users > Add user > Create new user*: email y contraseña (larga y única, de 12 caracteres o más) y marcá **"Auto Confirm User"**. Un disparador de la base le crea su perfil como empleado e inactivo.
3. **Promovelo a superadmin.** En el *SQL Editor* de Supabase, con el email en minúsculas (el comando está en el recuadro de abajo). Tiene que responder `UPDATE 1`. Después entrá a `/admin` con ese email y esa contraseña.

```
update mgdetailing.perfiles
set rol_id = 'superadmin', activo = true
where email = 'tu-email@ejemplo.com';
```

Cómo agregar un empleado

1. En Supabase: *Authentication > Users > Add user > Create new user*, con el email y una contraseña larga y única, y **"Auto Confirm User"** marcado.
2. Entrá al panel como superadmin y abrí la pestaña **Usuarios**: el empleado nuevo aparece en **"Pendientes de activación"** (como empleado inactivo).
3. Tocá **Activar**. Si tiene que ser admin o superadmin en vez de empleado, cambiale el **Rol** antes o después. Podés cargarle el nombre y tocar **Guardar**.
4. Pasale el email y la contraseña (por un canal privado). Con el rol **empleado** solo puede ver turnos y marcarlos como completados.
5. Para sacarle el acceso a alguien: **Desactivar** desde el panel (efecto inmediato) o borrarlo en *Authentication > Users*.
6. Contraseña olvidada: *Authentication > Users > (usuario) > Send password recovery*, o cambiarla ahí mismo.

Verlo desde el celular

1. Detené `npm run dev` y corré por separado `npm --prefix backend run dev` y `npm --prefix frontend run dev:lan`.
2. Vite muestra una dirección de red, por ejemplo `http://192.168.0.20:5173`.
3. En `backend/.env` agregá esa dirección a `FRONTEND_ORIGIN`, separada por coma, y reiniciá el backend.
4. Abrí esa dirección en el celular (misma Wi-Fi).

Verificar y publicar

- `npm run verificar` corre tipos, lint, tests y build de todo.
- Para publicar: `npm ci`, `npm run build` y `npm start`. Un solo proceso Node sirve la API y la web en el mismo dominio, detrás de HTTPS (Caddy o un PaaS), con `SERVE_FRONTEND=true`. En el repositorio no hay Dockerfile ni configuración de Caddy: son recomendaciones.

9. Cómo editar el contenido

Casi todo se edita en un puñado de archivos. Lo que queda vacío se oculta solo. Buscá el texto `COMPLETAR` en el código para ver todo lo que falta.

Qué querés cambiar	Archivo exacto	Cómo
WhatsApp, Instagram, dirección, horarios, link de Maps, dirección pública de la web	`frontend/src/config/sitio.ts`	Completá `whatsapp` (solo dígitos con código de país, ej. `'5493804123456'`), `instagram` (sin @), `direccion`, `horarios` (un renglón por ítem), `mapsUrl` (https) y `url` (sin barra final).
Servicios: descripción, qué incluye, duración, precio, destacado	`frontend/src/data/servicios.ts`	Para un precio, poné el número en pesos sin puntos: `precioDesde: 25000` se muestra "Desde $ 25.000". `null` muestra "Consultá".
Preguntas frecuentes	`frontend/src/data/preguntas.ts`	Editá, sumá o sacá preguntas.
Fotos de trabajos y antes/después	Fotos en `frontend/public/fotos/` y entradas en `frontend/src/data/galeria.ts`	Copiá la foto (JPG o WebP, unos 1600 px de ancho, menos de 400 KB) y agregá un objeto en `fotos` o en `antesDespues` con `src`, `alt` (qué se ve), `titulo` y `servicioId`. Si reemplazás una foto, cambiale el nombre: se guardan 7 días en caché.
Logo	`logo` en `frontend/src/config/sitio.ts`; archivos en `frontend/public/marca/`	El logo actual es `/marca/logo-fondo-oscuro.png`.
Archivos de marca (logo, monograma, imagen para redes, ícono)	Originales en `frontend/public/fotos/MgDetailing-*`	Después de cambiar un original, corré `npm --prefix frontend run marca` para regenerar las versiones livianas de `public/marca/`.
Textos fijos de cada sección (portada, cómo trabajamos, beneficios, contacto)	`frontend/src/landing/` (`Hero.tsx`, `Proceso.tsx`, `Beneficios.tsx`, `Contacto.tsx`, `Pie.tsx`)	Son archivos de código: conviene pedirle el cambio a quien mantiene el proyecto.
Colores y tipografía	`frontend/src/styles/` y `frontend/DESIGN.md`	Las fuentes están en `fuentes.css`.
Avisos, dominio y claves del servidor	`backend/.env`	A partir de `backend/.env.example`. Nunca al repositorio.
Un servicio nuevo	`frontend/src/data/servicios.ts`, `backend/src/domain/servicios.ts` y una migración nueva	Los tres tienen que coincidir o el backend no arranca.

> Después de editar, corré `npm run verificar` antes de publicar.

10. Pendientes del dueño

Checklist ordenada por prioridad. Marcá cada ítem cuando esté hecho.

Ya hecho

- Las dos migraciones de roles y perfiles están aplicadas en Supabase.
- El registro público de Supabase está cerrado.
- El superadmin está creado y el panel funciona (el dueño lo confirmó).

Prioridad 1: limpieza de seguridad

☐ **Borrar del** `backend/.env` las variables viejas, si todavía están: `SUPABASE_SECRET_KEY`, `JWT_SECRET` y `DB_DRIVER`. El backend no usa ninguna; la clave secreta es la más importante, porque da acceso total al proyecto. Si alguna vez se compartió, rotala desde el panel de Supabase.

☐ Usar contraseñas largas y únicas (12 caracteres o más; mejor 16) para todos los usuarios del panel.

Prioridad 2: datos personales y legales

☐ **Aviso de privacidad** en el formulario: elegir el nombre del responsable, el plazo de guardado y el canal para pedir acceso o borrado de datos.

☐ **Transferencia internacional a EE.UU.:** mencionar en el aviso que los datos se guardan en EE.UU. (región `us-east-1` de Supabase) y pedir el consentimiento del cliente.

☐ **DPA de Supabase** (acuerdo de tratamiento de datos): revisarlo y aceptarlo con ayuda de un abogado.

☐ **Plazo de retención:** aprobar o ajustar la propuesta (6 meses para completados y cancelados, 3 para pendientes viejos) y pedir que se implemente.

Prioridad 3: para salir al aire

☐ **Avisos de turnos:** configurar Telegram (gratis; pasos en la sección 7) y probar con un turno de prueba.

☐ **Hosting y dominio:** elegir dónde publicar (un VPS con Caddy o un PaaS) y el dominio, por ejemplo `mgdetailing.com.ar`. Después cargar `url` en `sitio.ts` y `FRONTEND_ORIGIN` y `PUBLIC_URL` en el backend.

☐ **Fotos de trabajos:** sacar fotos reales (sin fotos de stock ni generadas con IA) y cargarlas (sección 9).

☐ **Precios y duraciones:** cargar los precios y confirmar las duraciones y las descripciones de los servicios en `servicios.ts`. Confirmar que el Pulido no saca rayas profundas.

☐ **Datos del taller** en `sitio.ts`: WhatsApp (o decidir no mostrarlo), Instagram, dirección, horarios y Maps.

☐ **Backups (plan Free):** Supabase no ofrece copias descargables. Hacer un `pg_dump` propio una vez por semana y guardarlo cifrado y fuera de la PC.

☐ Leer las 10 **preguntas frecuentes** y corregir lo que no sea cierto.

☐ Aprobar el diseño (tipografía y monograma que gira), que ya están terminados.

☐ Decidir si el repositorio de GitHub es público o privado (verificar que `backend/.env` no esté versionado).

Pendientes técnicos menores (no los decide el dueño)

- Mover los originales de marca fuera de `public/` (S7), cookie con prefijo `__Host-` (S6, opcional), comprimir el video del monograma, actualizar el README del backend (todavía dice que dos migraciones están pendientes) y una nota vieja en `frontend/PRODUCT.md` que dice que el logo no existe.
- Más adelante: catálogo de servicios editable desde el panel (hoy los servicios están fijos en el código) y MFA opcional.

11. Equipo de agentes

El proyecto lo hizo un equipo de agentes de inteligencia artificial, cada uno con un rol y un modelo, coordinados por un orquestador, y el dueño aprobó el resultado. Los modelos son los de Anthropic: Opus para lo que necesita más criterio, Sonnet para lo rápido y sólido, Haiku para lo liviano.

Agente	Modelo	Qué hizo
Frontend	Opus	Diseño "cartel fileteado", landing, panel, adaptación del panel a los roles y pantalla de Usuarios.
Backend	Sonnet	API, repositorios, servicios, seguridad básica y avisos. Para los avisos de WhatsApp, Sonnet con esfuerzo medio.
Revisor de calidad y escalabilidad	Sonnet	Revisión de solo lectura en tres checkpoints (backend, tests, frontend).
Ciberseguridad	Opus	Auditoría 1 (informe 05) y correcciones.
Buscador de skills	Sonnet	Con `/find-skills` instaló la skill `typescript-security-review` para las auditorías.
Modelado de base de datos	Opus	Diseño de las tablas, RLS y rol de mínimo privilegio en Supabase.
Auth y roles	Opus	Login con Supabase Auth (BFF), roles superadmin, admin y empleado, gestión de usuarios y las correcciones de la auditoría 3.
Auditorías 2 y 3	Opus	Auditoría de la base de datos en Supabase y del login nuevo (hallazgos A1 a A13).
Documentación	Haiku y Sonnet	Los README, los informes de `reportes/` y este documento.

Reglas de trabajo: los agentes no leen `backend/.env` ni `backend/storage/`, y no hacen commits sin que se pida.

12. Glosario

Término	Qué significa
API	El servicio al que le piden cosas la web y el panel (guardar un turno, listar turnos).
Backend / frontend	Backend: la parte que corre en el servidor. Frontend: la parte que se ve en el navegador.
BFF	"Backend for frontend": el backend hace de intermediario y el navegador nunca habla directo con Supabase.
Landing	La página de inicio de la web pública.
React / Vite	React arma las pantallas; Vite prepara la web para publicarla.
Node.js / Express / TypeScript	Node corre JavaScript en el servidor; Express arma la API; TypeScript es JavaScript con tipos para detectar errores antes.
Supabase	Servicio en la nube que da la base de datos Postgres y el login (Supabase Auth).
Postgres	El tipo de base de datos que usa Supabase.
Esquema	Una "carpeta" de tablas dentro de la base. El del proyecto es `mgdetailing`.
Migración	Archivo SQL que cambia la estructura de la base.
SQL Editor	Pantalla de Supabase donde se pegan y ejecutan sentencias SQL.
RLS	Row Level Security: reglas de la base que dicen qué filas puede ver cada rol.
Rol de mínimo privilegio	Usuario de la base con solo los permisos que necesita (`mgdetailing_app`).
TLS	Cifrado de la conexión; "verificado" significa que se comprueba la identidad del servidor.
Data API	La puerta de Supabase que publica tablas por internet. Acá está cerrada para nuestro esquema.
Token / JWT	Comprobante firmado que prueba quién inició sesión.
Tope de sesión	Máximo de 12 horas desde el login: pasado ese tiempo hay que volver a entrar, aunque se haya usado el panel todo el rato.
`NO_ACCESS` / `FORBIDDEN`	Códigos de error del panel: el primero es "tu usuario no tiene acceso"; el segundo, "tu rol no puede hacer esto".
Supabase Auth	El servicio de login de Supabase: guarda los usuarios y las contraseñas.
Cookie `HttpOnly`	Cookie que JavaScript no puede leer, así que no se puede robar desde la página.
`SameSite=Strict`	La cookie no viaja en pedidos que vienen de otros sitios.
CSRF	Pedido falsificado entre sitios: otra web hace que tu navegador mande algo al panel sin que quieras.
CORS / Origin	Qué sitios pueden hablar con la API; el "origen" es el sitio desde donde viene el pedido.
Rate limit	Tope de intentos por IP para frenar la fuerza bruta y el spam.

Término	Qué significa
Fuerza bruta	Probar muchas contraseñas hasta acertar.
Honeypot	Campo oculto que solo llenan los bots.
Proxy	Intermediario (Caddy, nginx o el hosting) que recibe las conexiones de internet antes del servidor.
VPS / PaaS	VPS: un servidor que alquilás y administrás. PaaS: hosting que se encarga de más cosas.
HTTPS / HSTS	Conexión cifrada; HSTS le dice al navegador que use solo HTTPS.
Dominio	La dirección de la web (por ejemplo mgdetailing.com.ar).
Test	Prueba automática que comprueba que el código sigue funcionando.
Lint	Revisión automática del estilo y de errores comunes del código.
`npm audit`	Revisa si las librerías de terceros tienen vulnerabilidades conocidas.
SSRF	Que alguien use al servidor para pedir direcciones internas.
Slowloris	Ataque con conexiones muy lentas para agotar al servidor.
Advisors de Supabase	Chequeo automático de Supabase que avisa problemas de seguridad y rendimiento.
DPA	Data Processing Agreement: acuerdo que fija cómo un proveedor trata los datos personales.
Ley 25.326	Ley argentina de Protección de Datos Personales.
AAIP	Agencia de Acceso a la Información Pública, el organismo de control de esa ley.
Bot de Telegram	Cuenta automática de Telegram que manda los avisos de turnos.
WhatsApp Cloud API	Forma oficial de Meta para que un sistema mande mensajes de WhatsApp.
`.env`	Archivo con la configuración y los secretos del servidor. Nunca va al repositorio.
Repositorio / commit	El historial del código (en GitHub) y cada "foto" guardada de los cambios.
WCAG 2.1 AA	Pautas internacionales de accesibilidad web.
SEO / Open Graph	Datos para buscadores y para que el link se vea bien al compartirlo.
Filete / fileteado	Estilo argentino de ornamento pintado a mano, con líneas y volutas.